

Data And Modelling Report

Convolutional Neural Network Models for Speech Sentiment Analysis

Arsenii Rybchenko, Mykyta Huskov

September 2025

Executive summary:

This report compares three **Speech Emotion Recognition (SER)** models trained on **CREMA-D**, a dataset of *short acted clips* across genders and ethnic backgrounds with **six emotions** (*anger, disgust, fear, happiness, neutral, sadness*). We treat the task as a **6-class** multiclass classification problem.

We evaluate three **CNN** variants: (i) a **light baseline CNN**, (ii) a **CNN without attention**, and (iii) a **CNN with Convolutional Block Attention Module (CBAM)**. Audio is transformed into *fixed-size MFCC* tensors; *standard normalization, light time/frequency masking* and *random noise injection* are applied. Models are selected by **dev accuracy**.

On the **test set**, **CBAM** achieved **accuracy** = 66%, outperforming the no-CBAM CNN (63%) and the baseline (60%), with **macro-F1** = 66%. These results indicate that **attention** helps the CNN focus on *salient time-frequency regions* relevant to emotion cues.

We summarise **key hyperparameters**, training curves, and **confusion matrices**, and discuss when **CBAM** is most effective in this SER setup.

Inspired by Dal Rì et al. (2023), who compared CNNs with CBAM for **Speech Emotion Recognition** across several datasets, we run a **controlled ablation** on **CREMA-D**. We train two otherwise identical **CNNs**—with and without **CBAM**—on **MFCC** features and compare them using **dev and test accuracy/macro-F1** for reporting. This isolates the effect of **attention** under a *fixed feature set and training recipe*.

1 Data

We use the **CREMA-D** dataset Cao et al. (2014), an audio-visual corpus of **7,442** short clips from **91** actors speaking **12** sentences with **six** emotions (Anger, Disgust, Fear, Happy, Neutral, Sad) and **four** intensity levels (Low, Medium, High, Unspecified). Labels were obtained via **crowd-sourcing**: about **2,443** raters provided multiple judgments per clip,

and most clips received **7+** ratings; we use the majority label for supervised training. We work with the **audio** modality only.

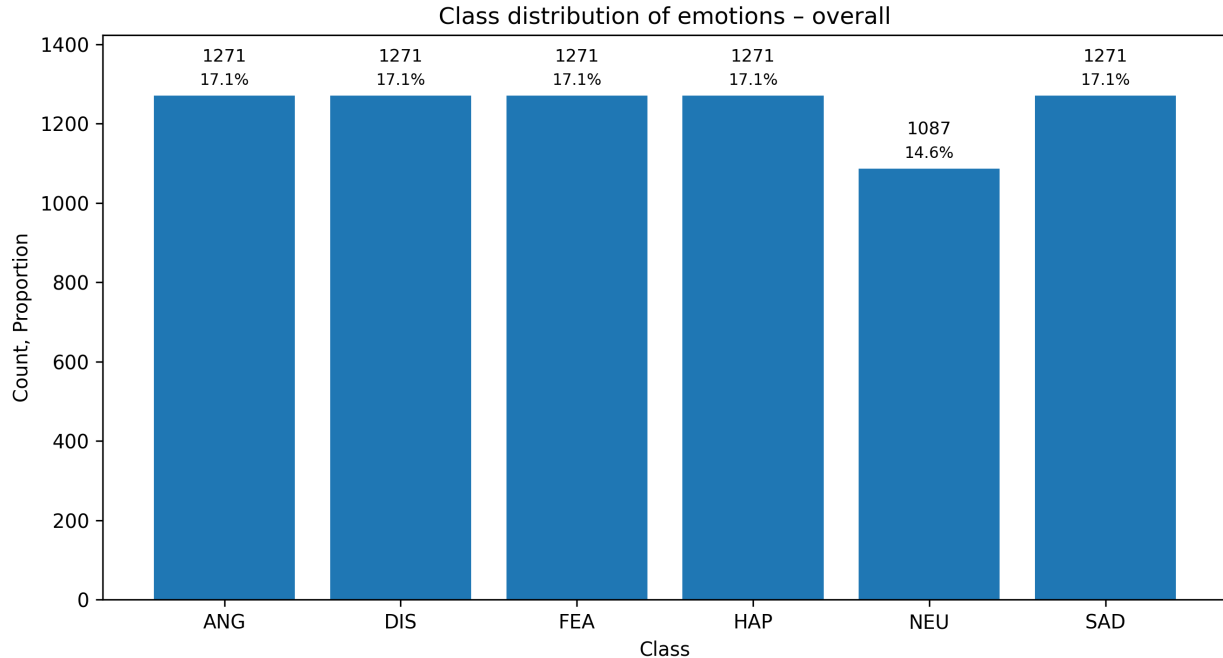


Figure 1: Class distribution of emotions in CREMA-D.

CREMA-D is suitable for **discrete Speech Emotion Recognition** due to its size and speaker diversity (gender and ethnicity).

1.1 Feature extraction and preprocessing

We *resample all audio to 16 kHz* and compute **MFCCs**. Each clip is split into *short frames* (`n_fft=512` $\approx$ 32 ms; `hop_length=256` $\approx$ 16 ms) with a *Hann* window. For every frame we make a **Mel power spectrogram**, convert it to a *log scale* (dB), and apply a **cosine transform (DCT)** to get **40 coefficients** (`n_mfcc=40`). We *ignore intensity levels*. The final feature map has shape $40 \times T$ and is reshaped to $1 \times 1 \times 40 \times T$ for the CNN.

For evaluation, we use an **80/10/10 train/dev/test** split that is **stratified by emotion** to keep class ratios balanced. The split is *random at the clip level* (not speaker- or sentence-independent). We also check *class counts per split* to watch for imbalance. This follows Dal Rì *et al.*, who also used random 80/10/10 splits and did not enforce speaker or sentence independence.

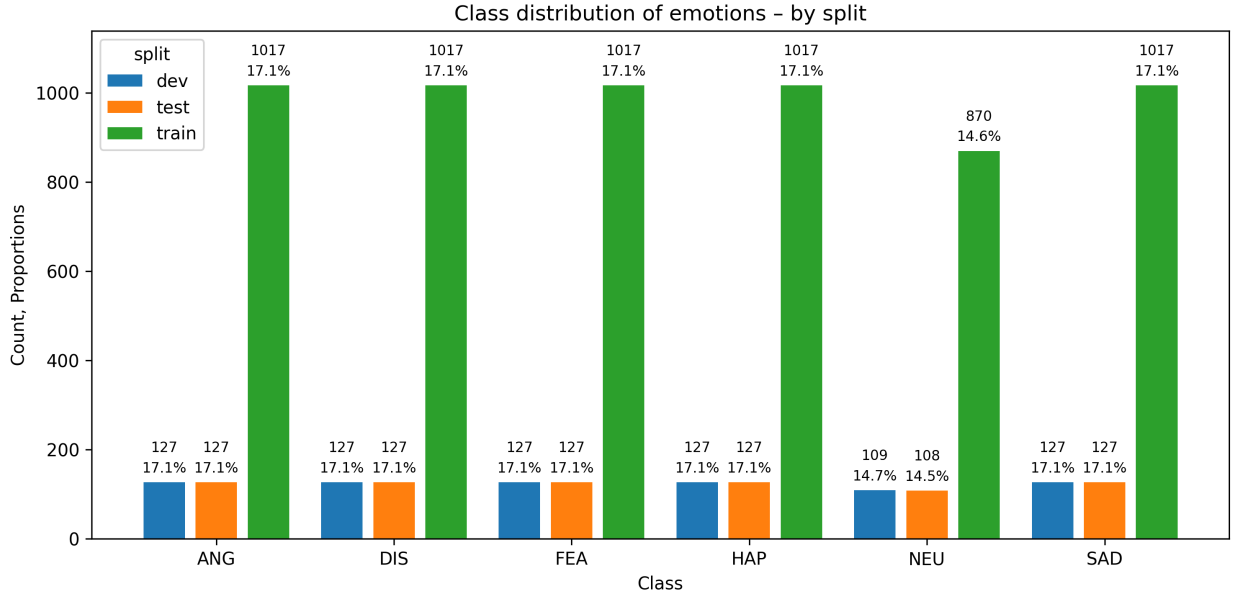


Figure 2: Class distribution of emotions in CREMA-D by train/dev/test split.

1.2 Data augmentation

To make the model more robust, we apply **data augmentation**. A *time shift* moves the MFCC features slightly left or right by a few frames. The empty part is either filled with zeros or wrapped around, which simulates speech starting a little earlier or later. This prevents the model from depending too much on the exact position of the signal.

We also add *random Gaussian noise*, scaled to the standard deviation of the signal, which mimics microphone hiss or background static.

Finally, we use **SpecAugment**: *frequency masking* randomly hides a band of frequency bins (like removing some tones), while *time masking* hides a short sequence of frames (like skipping a few milliseconds of speech). Both transformations force the network to learn from the remaining cues rather than memorising exact patterns.

Before these augmentations, each input tensor is either **padded or truncated** to a fixed length of **218 time frames**, ensuring that all samples have the same size for batch training.

Together, these steps help the model focus on the **essential emotion patterns** instead of overfitting to small details in the data.

2 Deep Learning modelling

2.1 Introduction

We frame the problem as a **multi-class classification task** with six emotion classes. To address it, we train and evaluate three **CNN-based models** under the same hyperparameter settings. These are: (i) a **baseline CNN**, (ii) a **CNN without attention**, and (iii) a **CNN with Convolutional Block Attention Module (CBAM)**. The comparison is designed as an **ablation study**, where the effect of adding attention is isolated. This setup is directly inspired by Dal Rì et al. (2023), who analysed the impact of CBAM on Speech Emotion Recognition across multiple datasets.

2.2 Hyperparameters and regularization

This section lists the **training hyperparameters** and the **regularization/selection** settings used in all runs. We train with minibatches of **24** using **Adam** to optimize **cross-entropy** (*label smoothing* = 0.05) at a learning rate of **0.001**. A **ReduceLROnPlateau** scheduler halves the learning rate ($\times 0.5$) when the *dev loss* does not improve for **4** epochs, with a floor at $0.01 \times \text{initial LR}$.

For regularization, we use **dropout** ($p=0.4$) in the classification head and **batch normalization** after each convolution. Model selection and early stopping monitor **dev accuracy** with **patience** = 10; the best checkpoint (by dev accuracy) is saved after epoch > 10.

To illustrate the training process, we present **accuracy curves** and **confusion matrices**. The accuracy plots show how performance evolves across epochs, while the confusion matrices highlight the **per-class recall**, i.e. how well the model distinguishes between different emotions.

Hyperparameters:

- **Batch size:** 24
- **Epochs:** 100
- **Optimizer:** *Adam*
- **Learning rate:** 0.001
- **LR scheduler:** **ReduceLROnPlateau** on *dev loss*; *factor*=0.5, *patience*=4, *min_lr*= $0.01 \times \text{LR}$;
- **Loss:** **CrossEntropyLoss**

Regularization:

- **Label smoothing:** 0.05 (**CrossEntropyLoss(label_smoothing)**)
- **Dropout:** 0.4 in the FC layer

- **Batch Normalization:** after each conv block (BatchNorm2d)
- **Weight init:** Xavier uniform for Conv2d/Linear; biases zeroed
- **Early stopping:** on *dev accuracy*; patience=10;
- **Reduction Ratio (CBAM only):** 16 (*the factor by which the number of features is reduced when flowing through CBAM*)

2.3 Baseline model

The baseline model is built from three repeated blocks with overall 127,686 parameters. Each block contains a **convolution layer**, followed by **batch normalization**, and then a **ReLU** activation function (**Conv** $\rightarrow$ **BN** $\rightarrow$ **ReLU**). After the three blocks, we apply **global average pooling** (GAP) over the time-frequency map and send the resulting vector into a small **classification head** (see Figure 3).

Input representation. We represent each audio file as **Mel-frequency cepstral coefficients** (MFCCs). These coefficients form a **two-dimensional map**: one axis is the **cepstral coefficient index**, and the other axis is **time**. Each position in this map holds a single **scalar value** that describes how strong a given coefficient is at a particular time step. Because there is only one value per position, the input has only **one channel**, similar to a grayscale image, whereas an RGB image has three (red, green, blue). In PyTorch notation, the full input tensor has shape $[N, 1, H, W]$, where N is the **batch size**, H is the **number of MFCC coefficients** (the height of the image), and W is the **number of time frames** (the width of the image, here padded to 218).

Convolution. The **convolution layer** scans the MFCC map with small **filters (kernels)**. Each filter looks at a **local neighborhood** in time and frequency, detecting small patterns such as **transitions** or **peaks**. The same filter is applied across the entire map, which reduces the number of parameters compared to a fully connected layer and makes the model more **data-efficient**. The first convolution also **increases the number of channels**, meaning it produces several **feature maps**; this allows the network to learn multiple types of local patterns in parallel.

Batch Normalization. After convolution, the activations are normalized by **batch normalization (BN)**. BN subtracts the **mean** and divides by the **standard deviation** of each channel, calculated over the current mini-batch, and then scales and shifts the result with learnable parameters (γ, β) . This keeps the activations in a **stable range**, prevents them from becoming *too large* or *too small*, and improves **gradient flow**. As a result, training is **faster**, **less sensitive** to initialization, and can use **higher learning rates** without diverging.

ReLU. The **rectified linear unit (ReLU)** introduces **nonlinearity** by setting all **negative values to zero** and leaving positive values unchanged. This has two main benefits: (1) it allows the model to learn more **complex, non-linear relationships** between features, and (2) it promotes **sparse activations**, since many outputs are zero, which can improve efficiency. ReLU also avoids the vanishing-gradient problems of older activation functions like sigmoid or tanh.

Layer order. The **order of the block** is important. By placing BN *between* the convolution and the activation (**Conv** $\rightarrow$ **BN** $\rightarrow$ **ReLU**), we normalize the **exact values** that will be passed into ReLU. This keeps most units **active** and reduces the risk of **dead ReLUs**. Because BN already provides a **shift parameter** β , the convolutional bias can be **disabled** (**bias=False**) to save parameters. Other orders, such as **Conv** $\rightarrow$ **ReLU** $\rightarrow$ **BN**, normalize the already-rectified outputs, making the statistics depend on which values were cut off by ReLU, which is **less stable** in practice.

Global average pooling. Finally, we use **global average pooling (GAP)**. This operation takes each **feature map (channel)** and computes the **average value** across all time and frequency positions. As a result, we get one scalar per channel. GAP summarizes the information in a **translation-tolerant way** (shifts in time do not change the average) and produces a **compact representation** without needing very large fully connected layers. The **classification head** then combines these channel values to predict the **emotion class**.

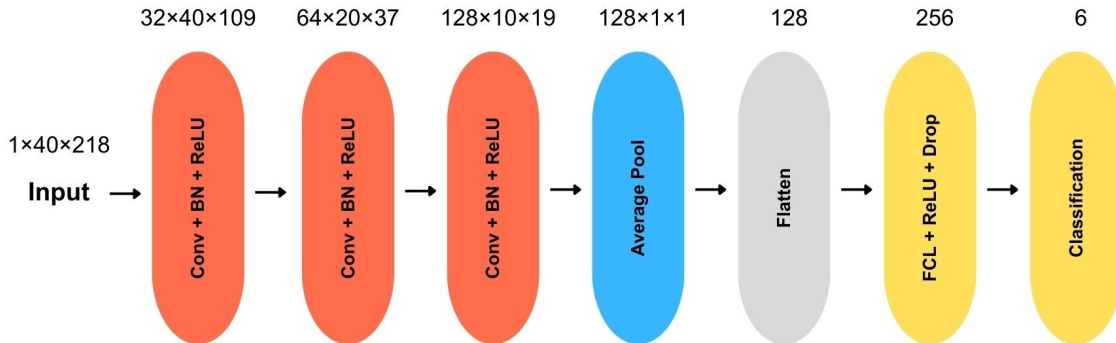


Figure 3: Baseline model architecture.

2.3.1 Training results

The **baseline model** was trained for the full 100 epochs, with *dev accuracy* gradually improving to **56%**. The **confusion matrix** reveals difficulties in distinguishing *disgust* and *fear*, with recalls of only **50%** and **40%**, respectively. Overall, the model provides a reasonable

starting point, reaching about **60%** accuracy on the test set.¹

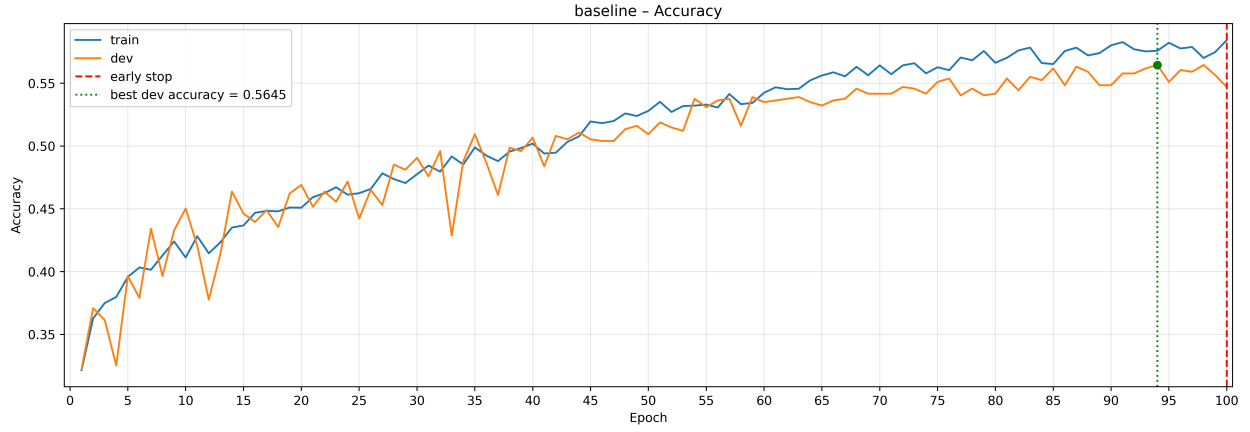


Figure 4: Baseline Accuracy.

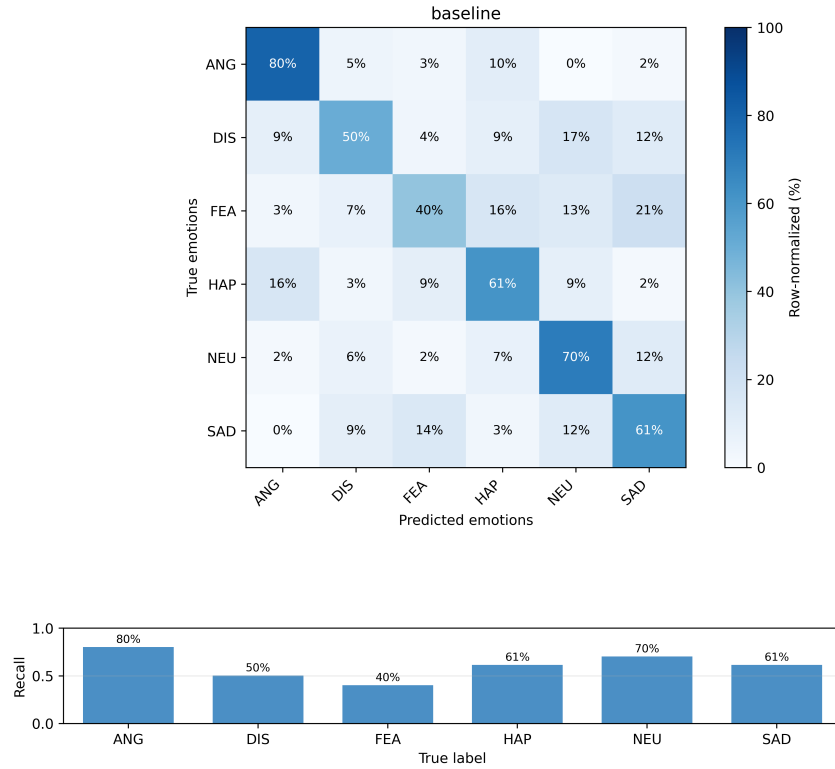


Figure 5: Confusion matrix and per class recall of baseline model.

¹This score may be inflated due to the lack of speaker-independent splits. The test set may contain speakers whose voices the model predicts more easily.

2.4 CNN with CBAM

The CNN with CBAM is a deeper model consisting of seven layers and approximately 1,881,945 learnable parameters² (see Figure 7). Apart from the CBAM, it introduces several enhancements: the use of the **GeLU** activation function instead of ReLU, the application of **dilated convolutions** to enlarge the receptive field, and the introduction of a **Max Pooling** layer in the middle of the network. Together with the CBAM attention module, these changes aim to increase the expressive power of the model for speech emotion recognition.

GELU. The **Gaussian Error Linear Unit (GELU)** is a smoother alternative to ReLU. While ReLU sets all negative activations to zero, GeLU allows small negative values to pass through, weighting them by the probability that a normally distributed variable is less than the input. For positive activations, GeLU slightly shrinks the values instead of leaving them unchanged (see Figure 6). This makes the activation function differentiable everywhere and better aligned with the assumption that pre-activations (after batch normalisation) are roughly Gaussian distributed.

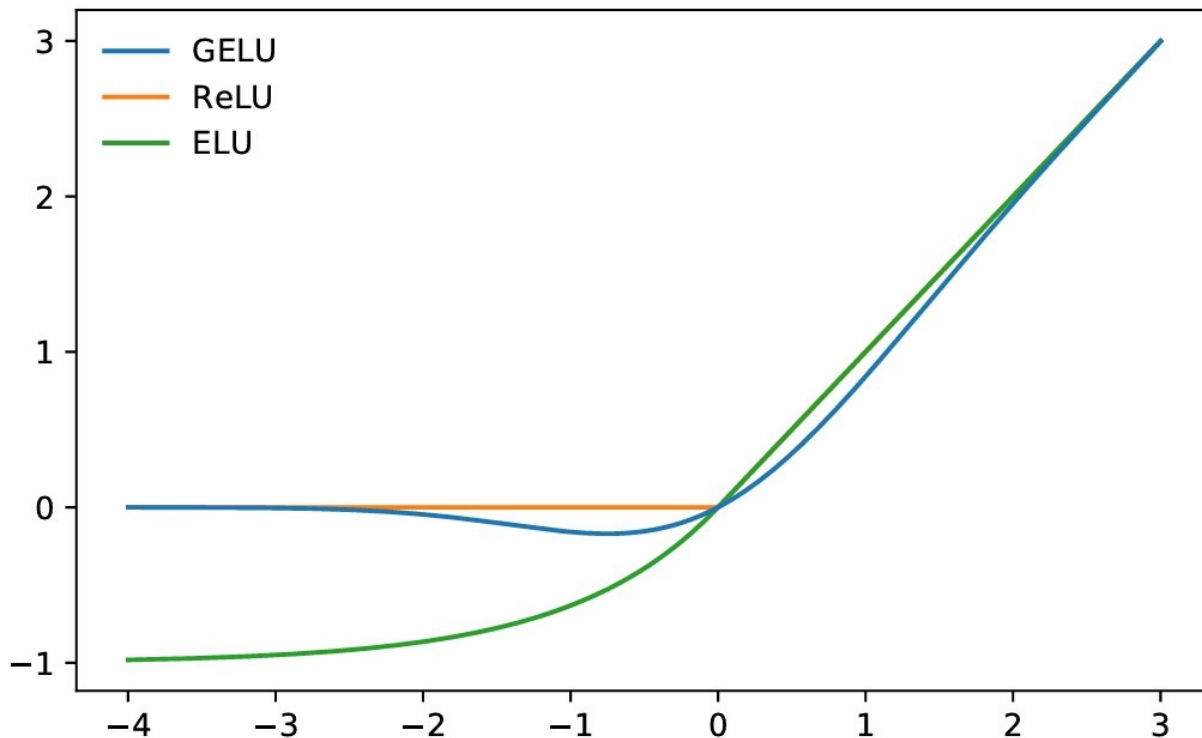


Figure 6: GELU vs ReLU (Hendrycks and Gimpel, 2016).

Dilation. **Dilated convolutions** increase the receptive field of a kernel by skipping over input elements with a fixed spacing. This allows the filter to capture information over a larger context window without adding more parameters. For speech-related tasks, this is valuable:

²CBAM amounts only to 8,563 parameters

emotions are not determined by a single frame but by patterns that span longer time spans. With dilation, the CNN can model such broader dependencies while still operating with compact kernels.

Max Pooling. The **Max Pooling layer** reduces the spatial dimensions of the feature maps by selecting only the strongest activation within each local neighborhood. In this architecture, the max pool step reduces the size from 7×7 to 6×6 , which might look like a small change, but its effect is important. First, it introduces a form of **local invariance**: small shifts in time or frequency (for example, when a phoneme occurs slightly earlier or later) do not change the result, since the maximum activation within the neighborhood is preserved. Second, it acts as a **noise suppressor**, discarding weaker, less relevant activations and keeping only the dominant signals. Third, max pooling **reduces computation** in the following layers, as subsequent convolutions operate on fewer spatial positions. Finally, it serves as a **shape control mechanism**: by reducing the size from 7×7 to 6×6 , the next convolution produces a clean 4×4 map that flattens neatly into 2048 features for the classification head. Thus, even though the dimensionality reduction is modest, max pooling plays multiple roles—robustness, efficiency, and architectural alignment.



Figure 7: CNN with CBAM and Linear Classification Head (Dal Rì et al., 2023).

CBAM CBAM is a lightweight module that can be inserted into a CNN block to help the network decide **what** to focus on and **where** to focus within a feature map Woo et al. (2018). It refines the intermediate representation in two sequential stages: **channel attention** and **spatial attention** (see Figure 8).

- **Channel Attention Module:** This stage focuses on *what* features are important across channels. Each channel corresponds to a feature detector (e.g., a spectral or temporal pattern in MFCCs). CBAM summarizes the activations in two ways: by *average pooling* (capturing overall presence) and by *max pooling* (capturing the strongest cues). Both descriptors are passed through a shared MLP, which produces a set of weights in the

range $[0, 1]$. Multiplying these weights with the feature map amplifies informative channels and suppresses less relevant ones.

- **Spatial Attention Module:** This stage focuses on *where* the most relevant information lies within the time–frequency map. After reweighting channels, CBAM compresses them by applying average and max pooling along the channel axis. The resulting two maps highlight potential regions of interest and are concatenated. A convolution layer with a 7×7 kernel then fuses these maps into a spatial attention mask. Multiplying this mask with the feature map makes the network emphasize salient regions (such as emotionally expressive time frames or frequency bands) and downplay noise or irrelevant areas.

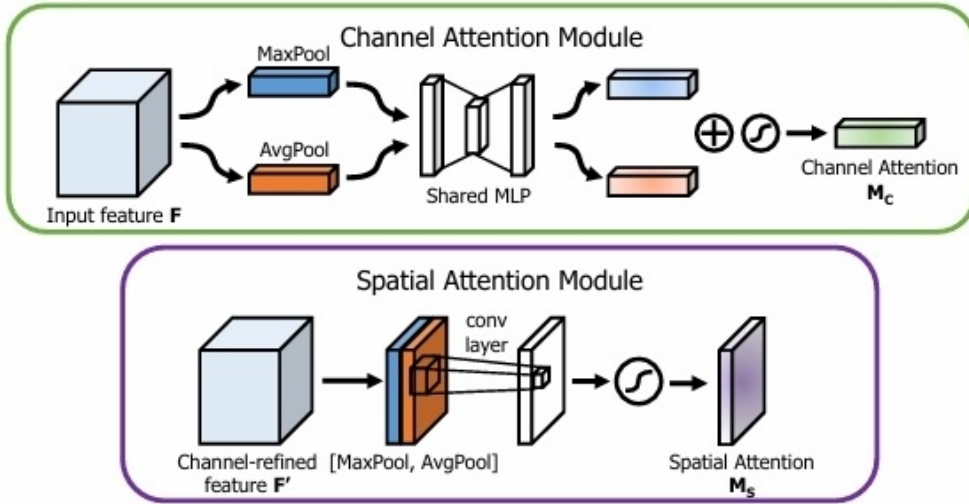


Figure 8: CBAM: Channel and Spatial Attention Modules (Woo et al., 2018).

2.4.1 Training results

The **CBAM model** inspired by Dal Rì et al. (2023) was trained for **69 epochs**, reaching a best *dev accuracy* of **66%**. The accuracy curves began to diverge after about **32 epochs**, indicating mild *overfitting*.³ Compared to the baseline, CBAM showed a much more stable **recall profile**, with the lowest scores observed for *fear* (50%) and *sadness* (59%). On the *test set*, the model achieved **66% accuracy**, surpassing the baseline by **6 percentage points**.

³This effect could likely be reduced with further hyperparameter tuning or stronger regularisation. In this study, we kept hyperparameters and seeds fixed to isolate the effect of CBAM.

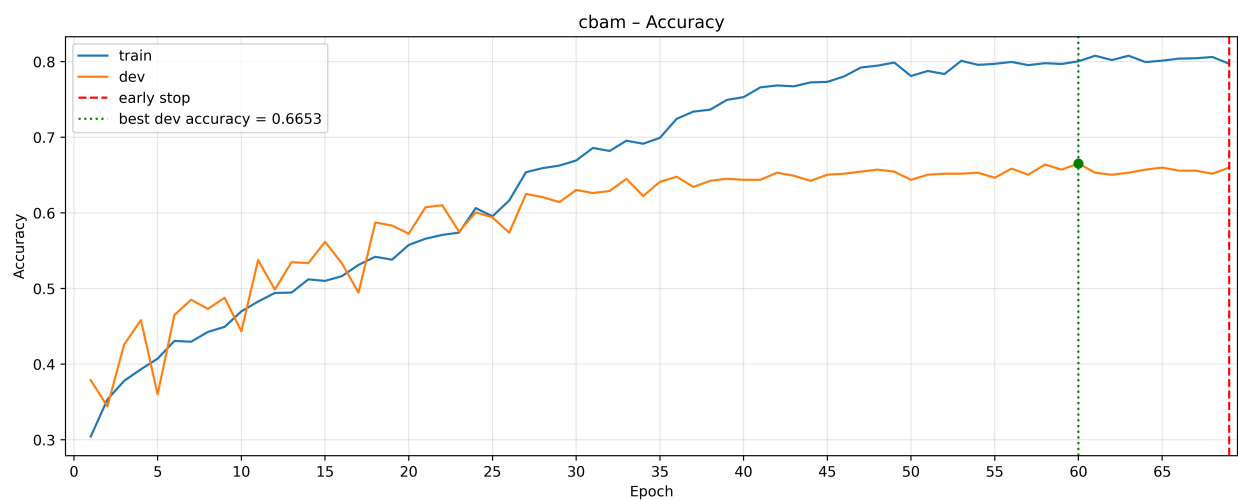


Figure 9: CBAM Accuracy.

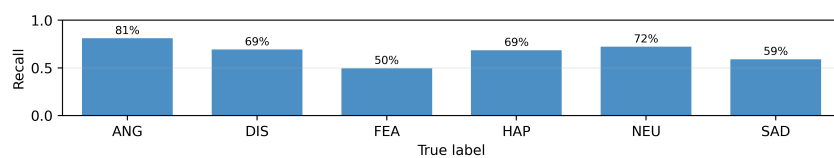
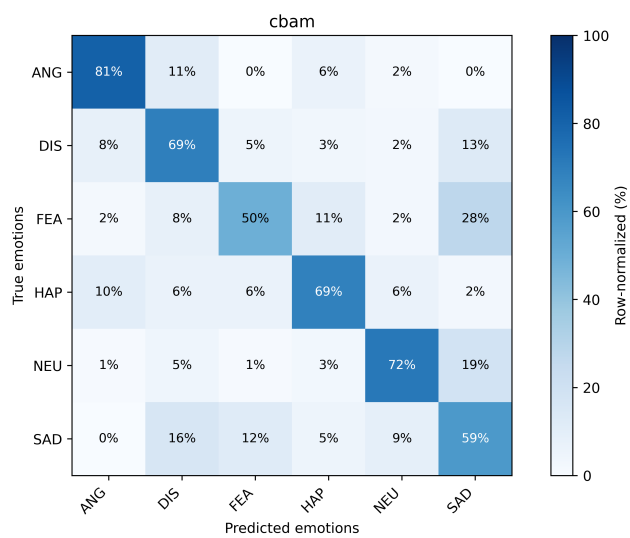


Figure 10: Confusion matrix and per-class recall of CBAM model.

2.5 CNN without CBAM

Because the two models differ only by the presence of **CBAM** (all other settings are identical), we therefore proceed directly to the **training results**.

2.5.1 Training results

The **CNN without CBAM** converged faster, reaching its best point by the **51st epoch**, and showed signs of *overfitting* around **32 epochs**, similar to the CBAM model. The best *dev accuracy* was **66%**, equal to CBAM. The **recall profile** was slightly more balanced overall, with the lowest recalls at **52%** and **57%**, compared to CBAM's lowest values of 50% and 59%. On the other hand, the highest recalls reached **76%** for the no-CBAM model and **81%** for CBAM. On the *test set*, the no-CBAM model achieved **63% accuracy**, underperforming CBAM by **3 percentage points**.

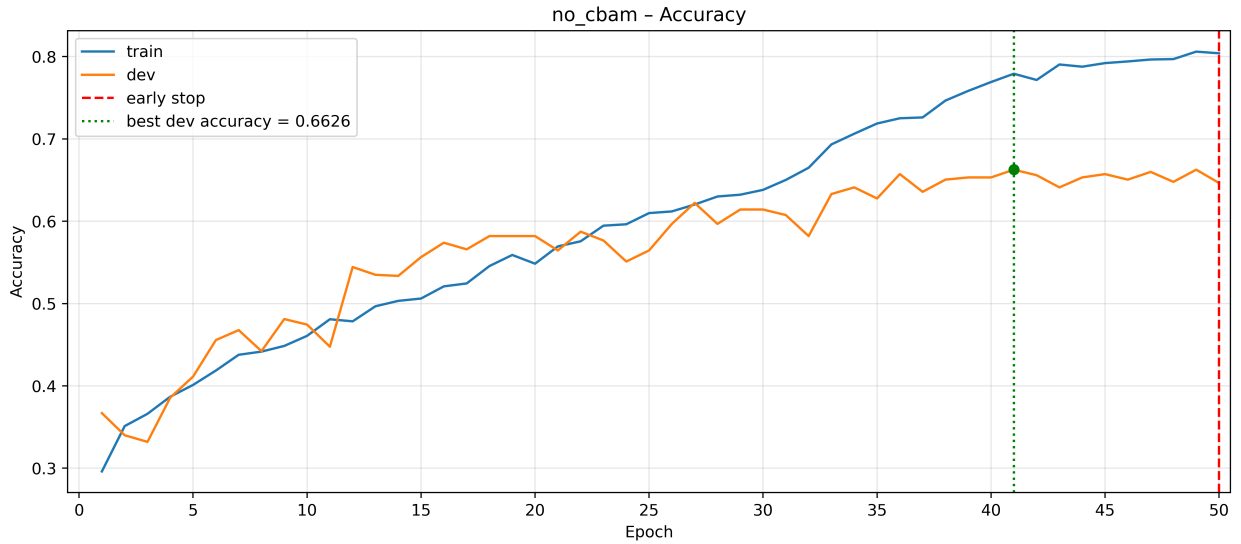


Figure 11: NO CBAM Accuracy.

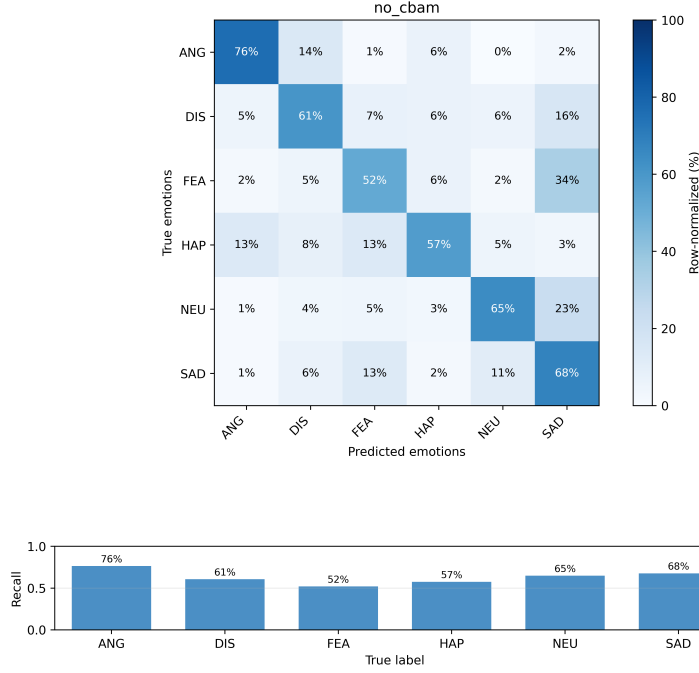


Figure 12: Confusion matrix and per class recall of NO CBAM model.

3 Conclusion

In conclusion, we will summarize the models' metrics, conclude, and state the study's limitations.

3.1 Comparative analysis

The following section will compare the performance of the CNN model we outlined with the Convolutional Block Attention Module and without, also showing the baseline results.

Table 1: Model comparison. Metrics are taken at the epoch with the best dev performance for each model.

Model	Accuracy			Macro-F1			Epochs
	<i>Train</i>	<i>Dev</i>	<i>Test</i>	<i>Train</i>	<i>Dev</i>	<i>Test</i>	<i>Early stop</i>
Baseline	0.5758	0.5645	0.6043	0.5734	0.5609	0.5993	-
NO CBAM	0.8060	0.6626	0.6312	0.8068	0.6631	0.6348	51
CBAM	0.8005	0.6653	0.6649	0.8008	0.6653	0.6665	70

The **Convolutional Block Attention Module (CBAM)** is designed to highlight the most informative parts of the input by focusing on relevant time-frequency regions. In the-

ory, this should improve both *convergence speed* and *final performance*. In our experiments, CBAM did not converge faster than the NO CBAM, but it did achieve stronger results: it matched the best **dev accuracy** of 66% (same as the no-CBAM CNN), while providing a more stable recall distribution across classes. On the **test set**, CBAM outperformed the baseline by **6 percentage points** and the no-CBAM model by **3 percentage points**.

Overall, we conclude that **CBAM is a beneficial extension** for Speech Emotion Recognition. With only $\approx 8,000$ additional parameters, the model gained a consistent **3% improvement in test accuracy** relative to its no-attention counterpart, while preserving the same training setup and hyperparameters. This suggests that attention helps CNNs capture subtle emotion cues more effectively without a major increase in model complexity.

3.2 Limitations

Also, we wish to highlight several points that could have been improved in the following study:

- **Insufficient runs for uncertainty.** Several reruns (e.g., >30 seeds) are needed to report stable confidence intervals and perform significance testing.
- **Emotion-based split.** train/dev/test split is designed stratified on the emotions which may bias the results due to the speaker imbalance in the splits.
- **Early-stopping on dev.** Tuning and checkpoint selection on the same dev set can induce selection bias. A dedicated validation would mitigate this.
- **Hyperparameter coverage.** LR schedules, batch size, and optimizer variants are only lightly tuned; results may be suboptimal.
- **External validity (cross-corpus).** Evaluation is on a single dataset; generalization to other datasets is not untested.

Sources

- Cao, H., Cooper, D. G., Keutmann, M. K., Gur, R. C., Nenkova, A., and Verma, R. (2014). Crema-d: Crowd-sourced emotional multimodal actors dataset. *IEEE Transactions on Affective Computing*, 5(4):377–390.
- Dal Rì, F. A., Ciardi, F. C., and Conci, N. (2023). Speech emotion recognition and deep learning: An extensive validation using convolutional neural networks. *IEEE Access*, 11:134190–134205.
- Hendrycks, D. and Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*. v5, revised 2023-06-06.

Woo, S., Park, J., Lee, J.-Y., and Kweon, I. S. (2018). CBAM: Convolutional Block Attention Module. In Ferrari, V., Hebert, M., Sminchisescu, C., and Weiss, Y., editors, *Computer Vision – ECCV 2018*, volume 11211 of *Lecture Notes in Computer Science*, pages 3–19. Springer, Cham.